

```
# Import PyTorch library (used mainly for deep learning & GPU handling)
import torch

# Import TensorFlow library (deep learning framework)
import tensorflow as tf

# Import Pandas for data manipulation (CSV, tables)
import pandas as pd

# Import NumPy for numerical computations
import numpy as np

# Import OpenCV for image processing tasks
import cv2

# Import Google Colab drive module to access Google Drive
from google.colab import drive

# ----- GPU CHECK -----

# Check whether GPU is available for PyTorch
print("GPU available:", torch.cuda.is_available())

# Print number of GPUs available
print("GPU count:", torch.cuda.device_count())

# If GPU is available, print the GPU name
if torch.cuda.is_available():
    print("GPU name:", torch.cuda.get_device_name(0))

# ----- GOOGLE DRIVE MOUNT -----

# Mount Google Drive to access large datasets stored in Drive
drive.mount('/content/drive')
```

```
GPU available: True
GPU count: 1
GPU name: Tesla T4
Mounted at /content/drive
```

```
# ----- INSTALL REQUIRED LIBRARIES -----
# Install PyTorch and related libraries with CUDA support (for GPU acceleration)
!pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118

# Install TensorFlow (deep learning framework)
!pip install tensorflow

# Install core data science libraries
!pip install pandas numpy scikit-learn

# Install OpenCV (headless version) for image processing tasks
!pip install opencv-python-headless # used for reading and processing images

# Install Librosa for audio signal processing
!pip install librosa # used for audio feature extraction

# Install Hugging Face Transformers for NLP tasks (BERT, GPT, etc.)
!pip install transformers # used for NLP and language models

# ----- GOOGLE DRIVE MOUNT (COLAB ONLY) -----
# Import Google Colab drive module
from google.colab import drive

# Mount Google Drive to access large datasets stored in Drive
drive.mount('/content/drive')

# ----- GPU AVAILABILITY CHECK -----
# Import PyTorch to check GPU support
import torch

# Check whether GPU is available
print("GPU available:", torch.cuda.is_available())

# If GPU is available, display the GPU name
if torch.cuda.is_available():
```

```
print("GPU name:", torch.cuda.get_device_name(0))
```

```
Looking in indexes: https://download.pytorch.org/whl/cu118
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.9.0+cu126)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.24.0+cu126)
Requirement already satisfied: torchaudio in /usr/local/lib/python3.12/dist-packages (2.9.0+cu126)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.20.3)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch) (12)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10)
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6)
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.0)
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch) (11)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch) (12)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.3.20 in /usr/local/lib/python3.12/dist-packages (from torch) (3.3.20)
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch) (12)
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch) (1.11)
Requirement already satisfied: triton==3.5.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.5.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (11.3.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.14.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch) (3.0.3)
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.19.0)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.6.3)
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.0)
Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<6.0.0dev,>=3.20.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.21.0)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.1)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.76.0)
Requirement already satisfied: tensorboard>=2.19.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.19.0)
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.10.0)
Requirement already satisfied: numpy<2.2.0,>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.15.1)
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow) (0.45.1)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.18.0)
```

TASK Explore different Deep Learning Packages and Install them

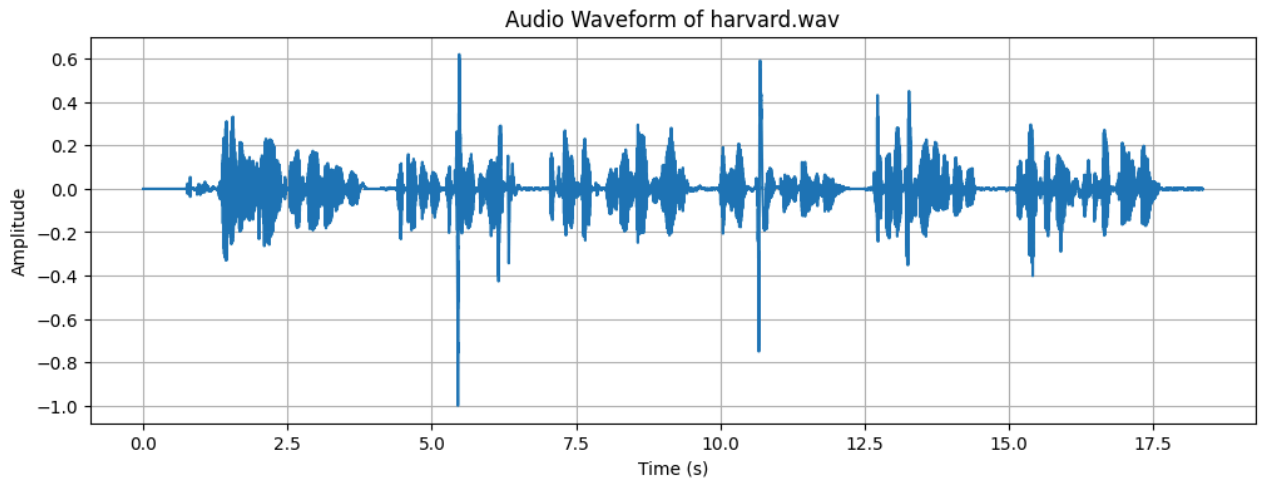
```
import keras
```

Start coding or [generate](#) with AI.

```
import matplotlib.pyplot as plt
import numpy as np

# Create a time axis for the audio waveform
time = np.linspace(0, len(audio_harvard) / sr_harvard, num=len(audio_harvard))

# Plot the audio waveform
plt.figure(figsize=(12, 4))
plt.plot(time, audio_harvard)
plt.title('Audio Waveform of harvard.wav')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.grid(True)
plt.show()
```



```
import librosa

# Load the audio file from the given path
# audio → 1D NumPy array containing the audio signal (amplitude values)
# sr → sample rate of the audio (samples per second)
# sr=None ensures the original sample rate is preserved
audio_harvard, sr_harvard = librosa.load("/content/harvard.wav", sr=None)

# Print the sample rate of the audio file
print("Sample rate for harvard.wav:", sr_harvard)

# Print the shape of the audio array
# Length indicates number of audio samples
print("Audio shape for harvard.wav:", audio_harvard.shape)
```

Sample rate for harvard.wav: 44100
Audio shape for harvard.wav: (809508,)

Start coding or [generate](#) with AI.

LOADING FILES IN DIFFERENT FORMATS

```
import pandas as pd

# Load CSV (most common tabular format)
df = pd.read_csv("/content/drive/MyDrive/data/student_lifestyle_100k.csv")
```

df

	Student_ID	Age	Gender	Department	CGPA	Sleep_Duration	Study_Hours	Social_Media_Hours	Physical_Activity	Stress
0	1001	22	Female	Science	3.50	7.3	3.3	3.4	114	
1	1002	20	Male	Engineering	2.72	5.5	7.2	6.0	142	
2	1003	20	Male	Medical	3.01	5.4	2.3	1.8	137	
3	1004	21	Male	Engineering	3.63	8.1	2.0	4.6	130	
4	1005	19	Male	Arts	3.14	6.8	2.6	4.3	4	
...	...	...	...	...	...	...	...	...	...	...
99995	100996	24	Female	Medical	2.02	7.0	0.6	1.4	66	
99996	100997	24	Male	Arts	2.33	5.0	3.6	5.2	103	
99997	100998	24	Female	Engineering	2.23	6.5	5.8	4.1	61	
99998	100999	19	Male	Engineering	3.61	6.1	4.8	4.9	116	
99999	101000	24	Female	Medical	2.10	7.9	3.5	4.0	79	

100000 rows × 11 columns

Task

✓ Load TSV (tab-separated)

```
df = pd.read_csv("/content/drive/MyDrive/data/train.tsv", sep="\t")
```

Load Excel (XLS/XLSX)

```
df = pd.read_excel("/content/drive/MyDrive/data/train.xlsx", sheet_name="Sheet1")
```

```
# Load regular JSON file
df = pd.read_json("/content/drive/MyDrive/data/nobel-prize-winners-by-year.json")
```

df

	year	winners
0	1901	[{'category': 'chemistry', 'winners': [{'name'...
1	1902	[{'category': 'chemistry', 'winners': [{'name'...
2	1903	[{'category': 'chemistry', 'winners': [{'name'...
3	1904	[{'category': 'chemistry', 'winners': [{'name'...
4	1905	[{'category': 'chemistry', 'winners': [{'name'...
...	...	...
110	2015	[{'category': 'chemistry', 'winners': [{'name'...
111	2016	[{'category': 'chemistry', 'winners': [{'name'...
112	2017	[{'category': 'chemistry', 'winners': [{'name'...
113	2018	[{'category': 'chemistry', 'winners': [{'name'...
114	2019	[{'category': 'chemistry', 'winners': [{'name'...

115 rows × 2 columns

```
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
df = pd.read_csv(url)
```

df

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

```
from sklearn.datasets import load_iris
X, y = load_iris(return_X_y=True)
```

```
from tensorflow.keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

TASK Upload built-in dataset

```
# Import OpenCV library (used for image processing; not directly used here)
import cv2
```

```
# Import PIL to load images in Python
```

```
from PIL import Image

# Import PyTorch library
import torch

# Import matplotlib for displaying images
import matplotlib.pyplot as plt

# Import torchvision transforms for image preprocessing
from torchvision import transforms

# -----
# STEP 1: Load the image using PIL
# -----

# Open the image file from the given path
pil_img = Image.open("/content/drive/MyDrive/data/cat.jpg")

# -----
# STEP 2: Define image transformations
# -----

# Resize image to 224x224 (standard input size for CNNs)
# Convert image to PyTorch tensor (values scaled to [0, 1])
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])

# -----
# STEP 3: Apply transforms to the image
# -----

# Apply preprocessing and convert PIL image to tensor
# Shape becomes: [3, 224, 224] → (Channels, Height, Width)
img_tensor = transform(pil_img)

# Add batch dimension required by deep learning models
# Final shape: [1, 3, 224, 224]
img_tensor = img_tensor.unsqueeze(0)

# -----
# STEP 4: Visualize the image
# -----

# Create a figure with fixed size
plt.figure(figsize=(6, 6))

# Remove batch dimension: [3, 224, 224]
# Change tensor order from (C, H, W) to (H, W, C)
# Convert tensor to NumPy array for matplotlib
img_np = img_tensor[0].permute(1, 2, 0).numpy()

# Display the image
plt.imshow(img_np)

# Hide axis values for cleaner visualization
plt.axis("off")

# Set title for the image
plt.title("Loaded image as tensor")

# Show the image
plt.show()
```

Loaded image as tensor



```
# Import librosa library for audio processing
import librosa

# -----
# STEP 1: Load the audio file
# -----

# Load the audio file from the given path
# audio → 1D NumPy array containing the audio signal (amplitude values)
# sr → sample rate of the audio (samples per second)
# sr=None ensures the original sample rate is preserved
audio, sr = librosa.load("/content/drive/MyDrive/data/EB0_anger_1.wav", sr=None)

# -----
# STEP 2: Display basic audio information
# -----

# Print the sample rate of the audio file
print("Sample rate:", sr)

# Print the shape of the audio array
# Length indicates number of audio samples
print("Audio shape:", audio.shape)
```

```
Sample rate: 24000
Audio shape: (47640,)
```

```
# Load entire text file
with open("/content/drive/MyDrive/data/sample.txt", "r", encoding="utf-8") as f:
    text = f.read()

# Load line-by-line
with open("/content/drive/MyDrive/data/sample.txt", "r", encoding="utf-8") as f:
    lines = f.readlines()
```

```
text
```

```
'The think tank of China's People's Liberation Army Rocket Force recently recruited 13 Chinese technicians\n from private c
ompanies, PLA Daily reported on Saturday.\nZhang Hao and 12 other science and technology experts received letters of appoin
tment at the founding ceremony of\nthe PLA Rocket Force national defense science and technology experts panel, according to
a report published by the\nPLA Daily on Saturday.\nHonored as "rocket force science and technology experts," Zhang and his
fellow experts from private companies will\nserve as members of the PLA Rocket Force think tank, which will conduct researc
h into fields like overall design of\nthe missiles, missile launching and network system technology for five years.\nThe ex
```

```
# -----
# SINGLE NEURON: FORWARD PASS ONLY
# Input → Weights → Sum → Activation → Output
# -----
```

```

import numpy as np

# Step 1: Input values
inputs = np.array([1, 2])    # x1 = 1, x2 = 2

# Step 2: Weights
weights = np.array([0.4, 0.6]) # w1, w2

# Step 3: Bias
bias = 0.2

# Step 4: Weighted sum
weighted_sum = np.dot(inputs, weights) + bias

# Step 5: Activation function (Sigmoid)
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Step 6: Output
output = sigmoid(weighted_sum)

# Step 7: Print each stage
print("Input:", inputs)
print("Weights:", weights)
print("Weighted Sum (z):", weighted_sum)
print("Final Output:", output)

```

```

Input: [1 2]
Weights: [0.4 0.6]
Weighted Sum (z): 1.8
Final Output: 0.8581489350995123

```

```

# -----
# BASIC NEURAL NETWORK IMPLEMENTATION
# -----

# Step 1: Import required library
import numpy as np

# Step 2: Define input data (AND logic)
# Each row is one input sample
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

# Step 3: Define expected output
# AND logic output
y = np.array([[0],
              [0],
              [0],
              [1]])

# Step 4: Initialize weights and bias randomly
# 2 input neurons -> 1 output neuron
weights = np.random.rand(2, 1)
bias = np.random.rand(1)

# Step 5: Define activation function (Sigmoid)
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Step 6: Define derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Step 7: Training parameters
learning_rate = 0.1
epochs = 10000

# Step 8: Training the neural network
for epoch in range(epochs):

    # Forward propagation
    z = np.dot(X, weights) + bias    # Weighted sum
    output = sigmoid(z)              # Activation output

    # Calculate error
    error = y - output

    # Backpropagation

```

```
d_output = error * sigmoid_derivative(output)

# Update weights and bias
weights += learning_rate * np.dot(X.T, d_output)
bias += learning_rate * np.sum(d_output)

# Step 9: Display final results
print("Final Weights:\n", weights)
print("Final Bias:\n", bias)

# Step 10: Test the network
print("\nPredicted Output after Training:")
print(output)
```

```
Final Weights:
[[5.4770905 ]
 [5.47709051]]
Final Bias:
[-8.30833826]

Predicted Output after Training:
[[2.46435717e-04]
 [5.56618327e-02]
 [5.56618323e-02]
 [9.33750640e-01]]
```

Task

Import PyTorch, TensorFlow, and Keras, then display their versions to confirm the deep learning environment setup.

Import Deep Learning Libraries

Subtask:

Import PyTorch, TensorFlow, and Keras to confirm their availability and readiness for use.

Check Library Versions

Subtask:

Display the versions of PyTorch, TensorFlow, and Keras to understand the installed package versions.

Reasoning: The subtask is to display the versions of PyTorch, TensorFlow, and Keras. I will use a code block to print the `__version__` attribute for each of these imported libraries.

```
import torch
import tensorflow as tf
import keras

print(f"PyTorch Version: {torch.__version__}")
print(f"TensorFlow Version: {tf.__version__}")
print(f"Keras Version: {keras.__version__}")
```

```
PyTorch Version: 2.9.0+cu126
TensorFlow Version: 2.19.0
Keras Version: 3.10.0
```

Reasoning: The previous code failed because `torch`, `tensorflow`, and `keras` were not defined, meaning they were not imported in the current execution context. To fix this, I need to import all three libraries before trying to access their `__version__` attributes.

```
import torch
import tensorflow as tf
import keras

print(f"PyTorch Version: {torch.__version__}")
print(f"TensorFlow Version: {tf.__version__}")
print(f"Keras Version: {keras.__version__}")
```

```
PyTorch Version: 2.9.0+cu126
TensorFlow Version: 2.19.0
Keras Version: 3.10.0
```


Final Task

Subtask:

Review the imported libraries and their versions to confirm the deep learning environment setup.

Summary:

Q&A

Yes, the deep learning environment setup has been confirmed with the specified libraries and their versions.

Data Analysis Key Findings

- Initially, attempts to display library versions failed due to the libraries not being imported into the execution environment.
- The deep learning libraries were successfully imported: `torch`, `tensorflow`, and `keras`.
- The versions of the installed deep learning libraries were successfully retrieved and displayed:
 - PyTorch Version: `2.9.0+cu126`
 - TensorFlow Version: `2.19.0`
 - Keras Version: `3.10.0`

Insights or Next Steps

- The deep learning environment is successfully set up with PyTorch, TensorFlow, and Keras, making it ready for developing and executing deep learning models.
- Verify these library versions meet the requirements for any specific deep learning projects to ensure compatibility and leverage desired features.